Cliff-sensor-based Low-level Obstacle Detection for a Wheeled Robot in an Indoor Environment

Conference Paper · January 2020

DOI: 10.1109/ICCE46568.2020.9042997

5 authors, including:

Sookhyun Yang
LG Electronics
11 PUBLICATIONS **47** CITATIONS

SEE PROFILE

Cliff-sensor-based Low-level Obstacle Detection for a Wheeled Robot in an Indoor Environment

Sookhyun Yang[+], Jung-Hwan Kim[+], Dongki Noh[+], Wonkeun Yang[+], and Minho Lee[+]
[+]LG Electronics Inc., Seoul, South Korea

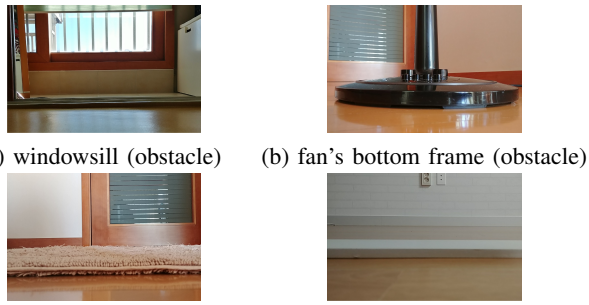
***Abstract*— A ramp and uneven ground formed by a low-level obstacle – whose height is too low from the ground – often stalls a robot's navigation in an indoor environment. Few-centimeter differences between a low-level obstacle and a low-level non-obstacle are very difficult to be precisely measured in a constant distance at a mobile robot. In this paper, a wheeled mobile robot thus makes physical contact onto a low-level object, in order to measure such subtle differences. We use one or more cliff sensors typically in place for a mobile robot in order to avoid a drop-off. A wheeled robot climbs over a low-level object, classifies an obstacle vs. a non-obstacle using a cliff sensor's timeseries data, and rapidly backs up before getting stuck onto an obstacle. While adopting a simplified deep-learning architecture, we suggest a rapid and accurate obstacle detection technique in real-time. We implemented our technique on an embedded robot platform of LG Hom-Bot. The supplementary video on the physical robot experiment can be accessed at https://youtu.be/yK57S857_II.**

I. Introduction

Over a couple of decades, robotics and vision communities developed numerous obstacle detection and avoidance techniques that employ various types of range sensors (such as Lidar [10], Radar [9], IR [6], sonar [12], and a camera sensor [5, 8, 11, 15]). Those techniques considered a wide set of target objects to detect and estimate their associated pose. Yet, a mobile robot in an indoor environment still suffers from a rough and uneven ground sporadically or permanently formed with low-level objects. As in Fig. 1, most low-level objects using a camera sensor shape like a horizontal line with a small amount of height difference. From observation at a distance, such an ambiguous dissimilarity between an obstacle and a non-obstacle is very difficult to be precisely and consistently measured to the same scale at a moving robot, which causes a robot to get stuck onto an obstacle or unnecessarily detour a non-obstacle.

In this paper, we develop a technique to make physical contact onto a low-level object for directly characterizing few-centimeter dissimilarities between a non-obstacle and an obstacle. We employ a cliff sensor *often found* and located on the bottom of a robot to detect a drop-off to avoid falls in a wheeled mobile robot [1, 2]. A cliff sensor reflects a signal (e.g., optical or ultrasonic) on the ground for measuring the distance to the ground. In our technique, a series of cliff-sensor data collected when a robot climbs over an object is used as a classification feature.

Our cliff-sensor-based technique has the following benefits. First, a cliff-sensor-based feature (i) is more distinctive by climbing over a target object at firsthand, (ii) is consistent by measuring the timeseries data on the verge of a robot's climbing over a target object in a constant distance, and (iii) consists of a few datapoints so that a classifier can be run on an embedded platform without heavy computing power. Second, since our technique is adoptable to an existing wheeled robot only with a SW module update, the technique significantly reduces the cost of production incurred for redesigning and manufacturing a new robotic product. Last, compared with other alternative approaches (e.g., a depth sensor [3]), a sensor typically used for cliff detection is very cheap (e.g., [4]).



(a) windowsill (obstacle) (b) fan's bottom frame (obstacle)
(c) carpet (non-obstacle) (d) door threshold (non-obstacle)

Fig. 1. RGB images of low-level objects from our physical robot (LG Hom-Bot) at the same distance

In spite of the above benefits, it is not trivial to identify a proper time at which a mobile robot stops climbing over an obstacle before getting stuck into an object too deeply. As more classifiable data available by more deeply climbing over an object, a robot is more likely to get stuck into an obstacle. We thus suggest real-time obstacle detection that reactively runs classification on the verge of a robot's climbing over an object. Also we adopt a simplified deep-learning architecture in order to support rapid and accurate classification as well as to be runnable on a resource-limited embedded platform. For an experimental purpose, we implement our technique on the LG Hom-Bot robotic vacuum cleaner – a canonical platform in indoor robotic applications.

The rest of this paper is organized as follows. Section II reviews related work. Section III abstracts a wheeled robot's navigation behavior from the perspective of one or more cliff sensors and designs our cliff-sensor-based obstacle detection technique. Section IV describes data collection and augmentation methodologies. Section V presents our data-analytic pipeline that uses a deep-learning algorithm, and evaluates the performance of our technique using real-world data. Section VI implements and demonstrates our technique

via a physical robot. Last, we conclude our paper.

II. Related Work

Numerous obstacle detection and avoidance solutions that detect the presence of an obstacle and estimate its associated position have been suggested using various sensors such as Lidar, Radar, IR, sonar, and camera sensors.

There are plenty of works that depended on an RGB or an RGB-D camera. Jafari et al. [8] suggested RGB-D based detectors that identify multiple persons and track their locations. In [11], they developed an obstacle detector that tracks relative size changes of multiple image frames. Broggi et al. [5] created a voxel-based map from a 3D point cloud to detect an obstacle and estimate its associated position and speed using a Kalman filter.

On the other hand, a number of works incorporated other types of sensors into cameras. Gageik et al. [6] used low-cost ultrasonic and IR sensors to detect an obstacle and also improved a distance to an obstacle with inertial and optical flow sensors data. In self-driving applications (e.g., [7]), obstacle detection with its associated position has been done by estimating depth information using a sequence of video frames while relying on wheel odometry. Nieuwenhuisen et al. [12] developed a multi-modal obstacle detector using multiple fisheye stereo cameras, a lightweight 3D laser scanner, and ultrasonic sensors. The recent work by Zhou et al. [15] used video frames and IMU (Inertial Measurement Unit) data to detect a very thin structured obstacle such as wires, cables and tree branches that a drone often crashes.

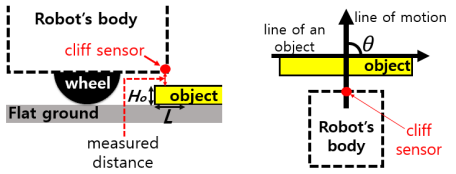
Unlike the prior works, we target to distinguish low-level objects whose height differences are in few centimeters, which are very difficult to be precisely and consistently measured at a distance. Our technique makes physical contact onto a target object to precisely characterize such subtle differences in a consistent manner.

III. Cliff-sensor-based Obstacle Detection

We begin by explaining the characteristics of timeseries data of a cliff sensor on a wheeled robotic platform in an indoor environment. Then we build a three-state model that abstracts a wheeled robot's **forward** navigation behavior against a low-level object with regard to the goodness of a classification feature. Given the three-state model, we design an obstacle-detection technique that reactively identifies the best timing to derive accurate classification.

A. Characteristics of cliff-sensor timeseries data in wheeled robot navigation

We consider a wheeled robotic platform that typically has one or more cliff sensors on its bottom. Suppose that time is equally divided into consecutive timeslots of length of Δt. At each timeslot t, each cliff sensor on a robot produces a datapoint that measures the distance to the ground or to a low-level object on the ground. A wheeled robot's cliff-sensor timeseries data on the flat ground shows a consistent distance with a negligible amount of deviation. On the other hand, when a wheeled robot climbs over an object from the ground, such distance would significantly get decreased or increased over multiple consecutive datapoints.



(a) side view (b) top view

Fig. 2. Schematic views of a wheeled robot (with one cliff sensor as an example) and a low-level object

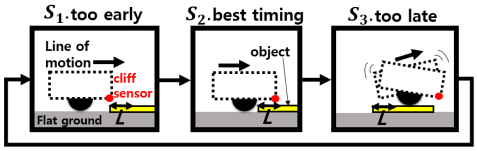


Fig. 3. Three sequential and circular states model

We parameterize a set of factors that play an important role in shaping cliff-sensor timeseries data as follows.

Classification feature N. Consider that our obstacle detection only uses the frontal part of an object in classification between an obstacle and a non-obstacle before a robot gets onto an obstacle too excessively. We let L be the *optimal* length of the frontal part of an object used to precisely characterize that object, as in Fig. 2a. However, a wheeled robot experiences frequent slips to climb over a low-level object, thus it is not trivial to precisely track whether a robot reaches up to L in length. Instead, we use an N-sized window that contains N consecutive cliff-sensor datapoints, all or some of which are presumably produced when a robot comes across an object and reaches up to L in length. Thus, we adopt those N consecutive datapoints in a window as a classification feature.

Height of an object H. We also let H be the frontal height of a climbing object (as in Fig. 2a) that allows a cliff sensor's timseries data to demonstrate the dissimilarities of low-level objects and that is also used to categorize an obstacle vs. a non-obstacle as we will see Section IV.

Robot's angle against an object θ. As in Fig. 2b, we let θ $[0^o \leq \theta \leq 180^o]$ an angle between the line of motion and the line of an object that characterizes how a robot approaches an object and thus differently shapes a pattern of one or more cliff-sensor data. We consider this factor in data augmentation as we will see Section IV.

B. Abstraction of a robot's forward navigation behavior

For simplicity, we abstract a robot's **forward** navigation behavior against a low-level object with the three-state model with regard to the goodness of a classification feature. While a robot moves forward in an indoor environment, the following three states sequentially circulate:

State S_1 as in Fig. 3. A robot navigates on the flat ground in an indoor environment or slightly comes across

an object with *too few* datapoints to distinctively characterize an object in an N-sized window. This poor feature causes a robot to often mis-classify a non-obstacle as an obstacle (as we will see Section V), which severely deteriorates a robot's navigation performance due to unnecessary non-obstacle avoidance. Thus, State S_1 is too early to result in accurate classification.

State S_2 as in Fig. 3. State S_2 is followed by State S_1. A robot is climbing over an object with sufficient datapoints to accurately characterize an object (in other words, a robot reaches up to L in length). Thus, State S_2 is appropriate to result in accurate classification.

State S_3 as in Fig. 3. State S_3 is followed by State S_2. In the case of a non-obstacle, a robot's entire body crosses over a non-obstacle, lands on the flat ground, and ends up being again in State S_1; otherwise, in the case of an obstacle, a robot's body gets stuck into an obstacle. At State S_3, a robot severely gets shaken and thus any data dissimilarities between a non-obstacle and an obstacle would be lost. Thus, State S_3 is too late to result in accurate classification.

C. Reactive obstacle detection

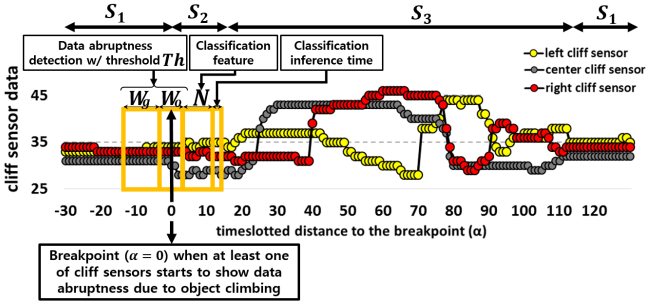


Fig. 4. Example of a timeseries plot with reactive obstacle-detection parameters of (N, W_g, W_o, Th) when a robot equipped with three cliff sensors climbs over a non-obstacle

As mentioned in our three-state model above, it is important to make a robot run classification only at State S_2 (NOT every timeslot). We note that one cliff sensor or at least one of multiple cliff sensors begins to show data abruptness across multiple consecutive datapoints when a robot climbs over an object. Our obstacle detection technique uses a moving time window to capture the starting timeslot of such data abruptness (we refer to the starting timeslot as a **breakpoint**) and reactively runs classification at State S_2. We let α be the timeslotted distance from a breakpoint. For instance, α=0 denotes a breakpoint. Before and after a breakpoint with a function of α, Fig. 4 shows an example of a three cliff-sensor timeseries plot with the parameters of our reactive obstacle detection (N, W_g, W_o, Th) below.

We let $\mathcal{W}_s(t) = \{X_{s1}, X_{s2}, \cdots, X_{sW}\}$ be a moving time window that consists of W most recent datapoints at timeslot t for cliff sensor s, where X_{si} is i-th datapoint out of W datapoints produced by cliff sensor s. At each timeslot t, our approach takes $\mathcal{W}_s(t)$ for cliff sensor s and investigates data abruptness using the equation (1) below.

$$\exists s, \Phi_s(t) = \left(\sum_{i=W_g+1}^{W} |X_{si} - \mu_s| \right) \geq Th, \text{ where } \mu_s = \frac{\sum_{i=1}^{W_g} X_{si}}{W_g} \quad (1)$$

We let W_g be the size of a window that consists of W_g consecutive datapoints produced when a robot stays on the flat ground, thus is used to judge whether a robot is indeed on the flat ground. We let W_o be the size of a window that consists of datapoints presumably produced on the verge of a robot's climbing over an object. We let $W = W_g + W_o$. We let Th be a threshold value that is a lower bound that determines whether a robot is on the State S_2. For each sensor s, $\Phi_s(t)$ quantifies the amount of data abruptness by comparing a mean value of past W_g datapoints with each of the remained W_o datapoints in a time window. Then the equation (1) evaluates whether at least one of cliff sensors shows significant abruptness equal to or greater than threshold Th. Once the equation (1) evaluates to True (i.e., concluding to believe that a robot starts to climb over a target object), a robot collects N more datapoints and then runs classification with those N datapoints while minimizing an inference time thus keep a sufficient time to back up from an obstacle, as we will see Section V.

IV. Data Collection and Augmentation

So far, we have introduced our cliff-sensor-based obstacle detection technique that operates in a function of parameters (N, W_g, W_o, Th). In this section, for validating the classifiability of few-centimeter differences over numerous low-level objects, we collect a large set of data that comprehensively characterizes a robot's reactive behavior in an indoor environment. We also synthesize more data from the collected data using our suggested data augmentation techniques. For an experimental purpose, we used the Hom-Bot platform [2]. The Hom-Bot as a wheeled robot has three cliff sensors located on its left-, center-, and right-bottom and runs on an embedded board with ARM 1176 Core and 512 MB memory.

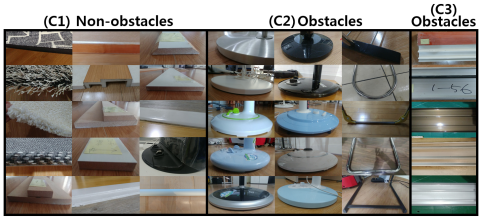


Fig. 5. 35 objects with different heights found in an indoor environment

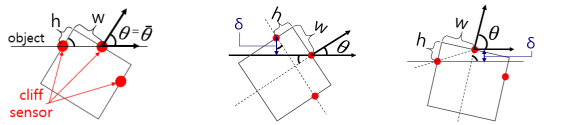
We arbitrarily categorize an obstacle vs. a non-obstacle as follows. Fig. 5 shows 35 representative objects with different heights ranging from > 0 to 3.5 cm with the classification categories below.

(**C1-non-obstacle**) consists of objects with $H \leq 1.5$ cm, considering the heights of low-level non-obstacle objects (e.g., a door threshold, a carpet) are around between > 0 and 1.5 cm.

(**C2-obstacle**) consists of objects with $H > 2$ cm (e.g., a fan's or a lamp's bottom frame). For blocking classification

between **(C1)** and **(C2)** from being obfuscated due to a small amount of height difference (i.e., 0.5 cm), we exclude an object with $1.5 < H \leq 2$ cm in our target object space, and focus on the classifiability between (**C1**) and (**C2**).

(**C3-obstacle**) consists of a windowsill that a wheeled robot is not subject to get onto and whose frontal part is different from (**C2**)'s, as shown in Fig. 5.



(a) $\overline{\theta} = \arctan(\frac{w}{h})$ (b) $0^o \leq \theta < \overline{\theta}$ (c) $\overline{\theta} < \theta \leq 90^o$

Fig. 6. Robot's navigation behaviors against an object with different angles. We only discuss $0^o \leq \theta \leq 90^o$. For $90^o < \theta \leq 180^o$, see the reflection symmetry below.

Having considered the above categories, we collect each three-state-model timeseries data by locating a robot at an arbitrary distance to an object and letting a robot continuously navigate from the flat ground to an object and from an object to the flat ground. We locate a robot with different values of θ, as in Fig. 6. We let w be the distance between a left sensor (or a right sensor) and a center sensor, and let h be the distance from a left sensor (or a right sensor) to the upper edge. Fig. 6a shows $\overline{\theta}[= \arctan(\frac{w}{h})]$-degree angle when both left and center sensors simultaneously climb over an object. Fig. 6b shows $0^o \leq \theta < \overline{\theta}$ when a left sensor first climbs over an object. Fig. 6c shows $\overline{\theta} < \theta \leq 90^o$ when a center sensor first climbs over an object.

We produced approximately 43,000 timeseries data over 35 objects. Since it is not easy to control a robot to exactly move in a particular angle, we roughly located a robot $0^o \leq \theta \leq 45^o$ and $45^o < \theta \leq 90^o$. For reducing data-collecting labor, we synthesize more data from the original dataset by employing two data augmentation techniques that synthetically generate timeseries data with more diverse values of θ as follows.

Reflection symmetry. We consider that multiple cliff sensors (other than a center cliff sensor) in a wheeled robot are often placed to be reflectively symmetric. Thus, we switch a left-sensor timeseries with a right-sensor timeseries, so that we produce synthetic data ranging in $90^o \leq \theta \leq 180^o$ from $0^o \leq \theta \leq 90^o$ original data.

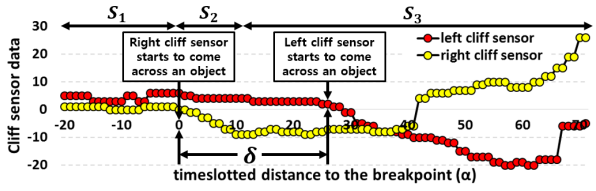


Fig. 7. Example of a timeseries plot with a shifting-data-augmentation parameter of δ when a robot equipped with two cliff sensors climbs over an obstacle

Shifting. As in Fig. 7's example, we let δ be the timeslotted distance from the timeslot when one of sensors is impacted first to the timeslot when another sensor is impacted next, denoted by a blue arrow as in Fig. 6b, c. δ can be quantified as follows. (Its derivation is found in [14].)

$$\delta = \begin{cases} (w - h \cdot \tan\theta) \cdot \cos\theta, & \text{if } 0 \leq \theta \leq \overline{\theta} \\ (h - \frac{w}{\tan\theta}) \cdot \sin\theta, & \text{if } \overline{\theta} < \theta \leq 90^o \end{cases} \quad (2)$$

As you see the above equation (2), we can synthetically produce timeseries data with different values of θ by simply increasing or decreasing the value of δ, i.e., shifting later-impacted sensors' timeseries data to the timeslots earlier or later than its original timeslot. This data augmentation method is extensible to different numbers of cliff sensors in the similar concept that considers there is a timeslotted distance between those sensors impacted by climbing over an object with different angles. (The data augmentation with two cliff sensors is found in [14].)

V. Data Analytic Pipeline and Validation

In this section, we first describe a deep-learning-based classification algorithm and its evaluation method, part of our data-analytic pipeline. Then we present our data-analytic pipeline that automates the workflow of determining the best-performing parameter values of cliff-sensor-based obstacle detection and validate our technique via the dataset collected in Section IV.

A. Deep-learning-based classification algorithm and its evaluation method

We describe the overall architecture of our classifier that employs a deep neural network. We design the network architecture of a deep learning algorithm to run classification on the embedded system in a short time, in order to keep a sufficient time to back up from an obstacle. Our neural network consists of two fully connected layers, each of which has 500 hidden units with ReLU (Rectified Linear Units) as an activation function. (Over the course of study in designing a deep learning algorithm, we found out that more layers than two, or more hidden units per layer than 500 did not give much improvement but increased an inference time.) Then the second fully connected layer is followed by one fully connected layer with a final log-softmax. On both the first and the second layers, we add batch normalization and dropout . The neural network takes as input N datapoints measured by three cliff sensors. Then the neural network returns four categories, **(C1)**, **(C2)**, **(C3)**, and the flat ground[1)].

We perform 10-fold cross validation using both the collected and the synthetically augmented data (consisting of 53,000 timeseries dataset), and discuss their average classification results in Section V-B.

Training. For training the neural network, we use a cross entropy as a loss function. We adopt the Adam optimizer with a mini-batch size of 100. We also use early stopping that stores a copy of the model parameters that improves the validation set error over the course of training and terminates when no parameters have improved over the best recorded

[1)]in case of a robot's running classification on the flat ground at State S_1 due to instantaneous glitches

validation error for some predefined number of iterations. For each timeseries data, we cropped next N datapoints produced exactly from the breakpoint (α = 0). Over the course of our study, we investigated N datapoints cropped at different α values and observed that N datapoints cropped from $\alpha = 0$ showed the best classification accuracy.

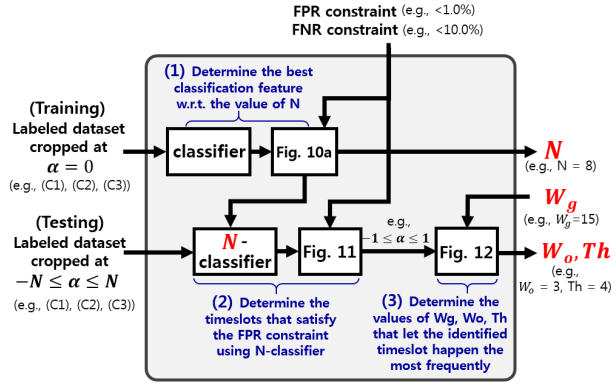
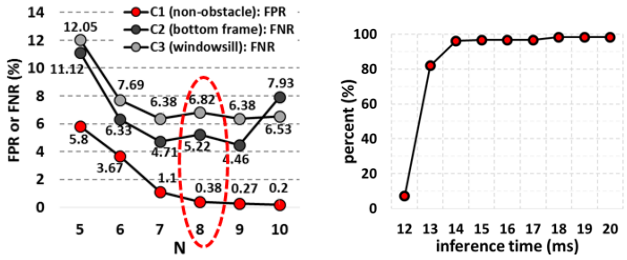


Fig. 8. Data-analytic pipeline



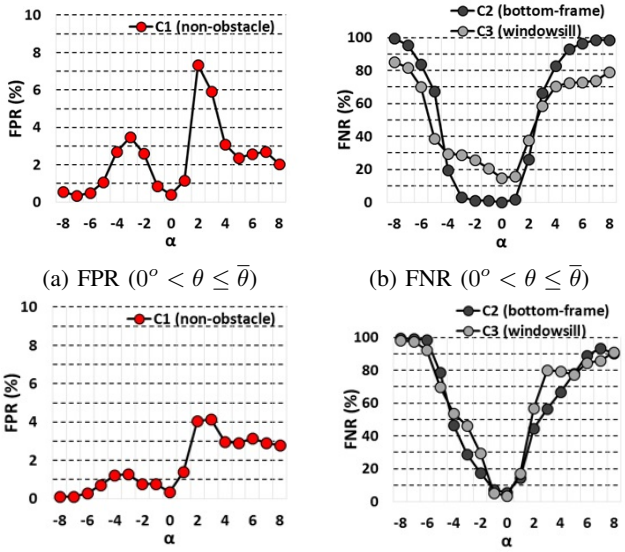
(a) FPR and FNR (b) CDF of an inference time

Fig. 9. Classification results and their inference times with N=8.

Testing. Regardless of data abruptness detection as in Section III-C, a robot in the middle of moving tends to run classification *not exactly at a breakpoint*. For mimicking such behavior of a moving robot, we crop a set of next N datapoints produced from different values of α for each timeseries; for instance, $-N \le \alpha \le N$ that consists of the timeslots at which N datapoints as a classification feature contains at least one datapoint produced when a robot climbs over an object.

Performance metrics. We quantify classification performance using the FPR (False positive rate) and the FNR (False negative rate). We only show FPR and FNR, since TNR (True negative rate) equals to 1 - FPR, and TPR (True positive rate) equals to 1 - FNR. The FPR denotes the fraction of cases where a non-obstacle is classified as an obstacle given that it is actually a non-obstacle. The FNR denotes the fraction of cases where an obstacle is classified as a non-obstacle given that it is actually an obstacle. If the FPR were to be high, the classifier would wrongly back-up from a non-obstacle and severely deteriorate navigation performance. Thus, for our purposes here, we consider it as an acceptable result when the FPR is less than **1.0%**, and when the FNR is less than **10%**.

B. Data-analytic pipeline of determining the best-performing parameter values of (N, W_g, W_o, Th)



(a) FPR ($0^o < \theta \le \bar{\theta}$) (b) FNR ($0^o < \theta \le \bar{\theta}$)

(c) FPR ($\bar{\theta} \le \theta \le 90^o$) (d) FNR ($\bar{\theta} \le \theta \le 90^o$)

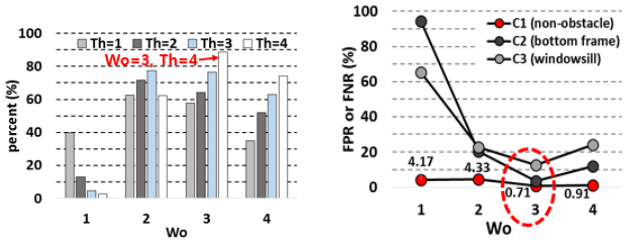
Fig. 10. $N = 8$ results with the test data cropped at $-8 \le \alpha \le 8$

Given FPR and FNR constraints, Fig. 8 shows our data-analytic pipeline that determines the best-performing parameter values of (N, W_g, W_o, Th).

(1). We first identify the optimal value of N. Let us begin our discussion of the classification results when $N = 5, 6, 7, 8, 9, 10$, given the dataset cropped at α = 0. (We do not consider $N > 10$ when a robot gets onto an object too deeply.) Fig. 9a plots the cross-validated FPR results of (**C1**), the cross-validated FNR results of (**C2**) and (**C3**) on average. As in Fig. 9a, the red-dotted curve for (**C1**) shows that our classifier guarantees the FPR less than 1% once $N \ge 8$, and other curves for (**C2**) and (**C3**) show the FNRs less than 10% once $N \ge 6$. We thus choose N = 8 that shows acceptable results and minimally gets onto an object as our best-performing classification features. And Fig. 9b plots the CDF of an inference time for which a classifier with N = 8 walks through our suggested network architecture and returns a classification result. Fig. 9b shows that a classifier with N = 8 runs mostly in 14 ms and in the median inference time of 12 ms.

(2). Having considered N = 8 as the best-performing classification features, we then evaluate the performance of (N = 8)-classifier using the (**C1**), (**C2**), and (**C3**) test datasets cropped at $-8 \le \alpha \le 8$; for instance, (α = -8)-cropped series means that a robot reactively detects the presence of a target object at α = -9 and considers next 8 datapoints as classification features. Fig. 10a, b, c, and d plot the FPR and the FNR results of the classifier with $0^o \le \theta \le \bar{\theta}$ and $\bar{\theta} \le \theta \le 90^o$ on average, respectively. As in Fig. 10, all FPR and FNR curves importantly show that the test data cropped only around $-1 \le \alpha \le 1$ guarantees the FPR less than 1% and the FNR (aggregate over (**C2**) and (**C3**)) less than 10%.

(3). Then how can we determine the parameter values



(a) Best values of W_o, Th for $-1 \leq \alpha \leq 1$

(b) Expected FPR and FNR with W_o = 3, Th = 4

Fig. 11. Best values of W_o, Th for running classification at $-1 \leq \alpha \leq 1$

of W_o, W_g, Th for reactively running our classifier around $-1 \leq \alpha \leq 1$? We first choose sufficient datapoints (e.g., W_g = 15) that well represent the flat ground. For different values of W_o and Th, we then use the equation (1) and identify the timeslots (in α) at which classification indeed happens with those given parameter values for each series. Then, for each parameter values, we quantify the fraction of the cases that run a classifier at $-1 \leq \alpha \leq 1$ over all the cases, as in Fig. 11a. Also we show the expected classification accuracy for each parameter values, as in Fig. 11b. Last, we choose the values of W_o, Th that satisfy the FPR and FNR constraints.

Fig. 11a plots the fractions of the cases of running a classifier at $-1 \leq \alpha \leq 1$ over all the cases for $W_o = 1, 2, 3, 4$ and $Th = 1, 2, 3, 4$. (We do not consider $W_o > 4$ and $Th > 4$ which are cases when a robot gets into an object too deeply.) $W_o = 3$ and $Th = 4$ show the highest fraction out of all parameter values and show approximately 90% of the cases run a classifier at $-1 \leq \alpha \leq 1$. Fig. 11b shows that having $W_o = 3$ and $Th = 4$ supports the FPRs less than 1% (0.71%) and the lowest FNRs (3.56% for (**C2**) and 12.7% for (**C3**)) – less than 10% in aggregate.

VI. Physical Robot Experiment

We implement our suggested technique in the Hom-Bot and demonstrate real robot navigation using the objects not considered for training. The Hom-Bot runs on our own robot platform that is conceptually similar to ROS (Robot operating system) [13] and allows multiple processes to exchange messages based on the publish-subscribe model. While monitoring a buffer that collects data measured from three cliff sensors every timeslot, our detection and classification process sends a message to avoid an obstacle to the navigation process that governs the movement of the Hom-Bot. Fig. 12 shows a series of snapshots in which the Hom-Bot successfully avoids cables, a lamp's bottom frame, and a windowsill, respectively. More videos on obstacle avoidance are found at https://youtu.be/yK57S857_II.

VII. Conclusion

In this paper, we developed cliff-sensor-based low-level obstacle detection that distinguished an obstacle by climbing over an obstacle for a wheeled robot in an indoor environment. We presented the data-analytic pipeline that adopted a deep-learning algorithm and quantitatively determined the best performing parameters of the obstacle detection, while guaranteeing a sufficient time to escape an obstacle even after climbing over an obstacle. Then we implemented and demonstrated our cliff-sensor-based obstacle detection on an embedded system.

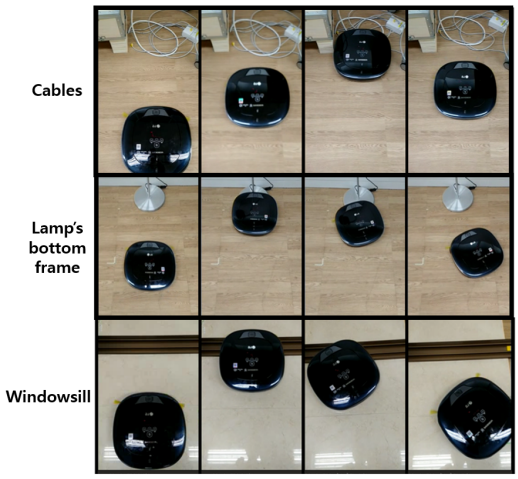


Fig. 12. A series of snapshots on the LG Hom-Bot's obstacle avoidance: cables, a lamp's bottom frame, and a windowsill

References

[1] iRobot Roomba, https://www.explainthatstuff.com/how-roomba-works.html. 2018.

[2] LG Hom-Bot, https://en.wikipedia.org/wiki/LG_Roboking. 2018.

[3] Intel® RealSense™ Depth Camera D435, https://click.intel.com/intelr-realsensetm-depth-camera-d435.html. 2019.

[4] PSD Sharp 5SK, https://www.digikey.com/product-detail/en/sharp-socle-technology/GP2Y0A51SK0F/1855-1027-ND/4103863. 2019.

[5] A. Broggi et al. A full-3d voxel-based dynamic obstacle detection for urban scenario using stereo vision. In *16th International IEEE ITSC 2013*, pages 71–76, Oct 2013.

[6] N. Gageik, P. Benz, and S. Montenegro. Obstacle detection and collision avoidance for a uav with complementary low-cost sensors. *IEEE Access*, 3:599–609, 2015.

[7] C. Häne, T. Sattler, and M. Pollefeys. Obstacle detection for self-driving cars using only monocular cameras and wheel odometry. In *2015 IEEE/RSJ IROS*, pages 5101–5108, Sept 2015.

[8] O. H. Jafari, D. Mitzel, and B. Leibe. Real-time rgb-d based people detection and tracking for mobile robots and head-worn cameras. In *2014 IEEE ICRA*, pages 5636–5643, May 2014.

[9] T. Kato, Y. Ninomiya, and I. Masaki. An obstacle detection method by fusion of radar and motion stereo. *IEEE Transactions on Intelligent Transportation Systems*, 3(3):182–188, Sept 2002.

[10] D. Maturana and S. Scherer. 3d convolutional neural networks for landing zone detection from lidar. In *2015 IEEE ICRA*, pages 3471–3478, May 2015.

[11] T. Mori and S. Scherer. First results in detecting and avoiding frontal obstacles from a monocular camera for micro unmanned aerial vehicles. In *ICRA*, May 2013.

[12] M. Nieuwenhuisen et al. Multimodal obstacle detection and collision avoidance for micro aerial vehicles. In *2013 European Conference on Mobile Robots*, pages 7–12, Sept 2013.

[13] M. Quigley et al. Ros: an open-source robot operating system. In *ICRA Workshop on Open Source Software*, 2009.

[14] S. Yang, J.-H. Kim, D. Noh, W. Yang, and M. Lee. Cliff-sensor-based low-level obstacle detection for a wheeled robot in an inddor environment. In *LG Electronics technical report*, 2019.

[15] C. Zhou, J. Yang, C. Zhao, and G. Hua. Fast, accurate thin-structure obstacle detection for autonomous mobile robots. In *2017 IEEE CVPRW*, pages 318–327, July 2017.